

ECR & ECS with Fargate

강사 정철웅 멘토
[cwjung123@gmail.com]

Version 2.7

[참조가이드]

ECR 참조

https://docs.aws.amazon.com/ko_kr/AmazonECR/latest/userguide/what-is-ecr.html

ECS 참조

<https://docs.aws.amazon.com/AmazonECS/latest/developerguide/Welcome.html>

API-Guide

https://docs.aws.amazon.com/ko_kr/AmazonECS/latest/APIReference/Welcome.html

ECS-CLI

<https://docs.aws.amazon.com/cli/latest/reference/ecs/index.html>

[ECS 개념이해]

<https://docs.aws.amazon.com/AmazonECS/latest/developerguide/create-container-image.html>

[ECS 설정인터페이스]

- Amazon ECS 콘솔
- AWS Command Line Interface (AWS CLI)
- AWS SDK
- AWS Copilot

[ECS-Task 정의]

▶ Why? "ECS +Fargate"

AWS 환경에서 신규 컨테이너 기반 서비스를 구축할 경우, Amazon ECS 와 AWS Fargate 를 조합한 아키텍처가 가장 권장되는 운영 방식 중 하나로 평가된다.

ECS + Fargate 구성은 서버 및 노드 관리 부담 없이 컨테이너를 실행할 수 있으며, Auto Scaling, IAM, ALB, CloudWatch, Aurora 등 AWS 핵심 서비스와 높은 수준으로 통합된다.

특히 Kubernetes(EKS) 대비 운영 복잡도가 낮고 구축 속도가 빠르기 때문에, 일반적인 웹 서비스·API 서비스·마이크로서비스 환경에서는 AWS 의 대표적인 실무형 컨테이너 아키텍처로 널리 사용된다.

▶ Task 제약사항 for AWS Fargate

- 운영 체제 : Amazon Linux 2, Windows Server 2019&2022 --> 실무에서는 대부분 Linux Fargate 사용
- CPU 아키텍처 : ARM, X86_64

▶ Task CPU 및 메모리 for AWS Fargate

256(.25 vCPU)	512MiB, 1GB, 2GB	Linux
512(.5 vCPU)	1GB, 2GB, 3GB, 4GB	Linux
1024(1 vCPU)	2GB, 3GB, 4GB, 5GB, 6GB, 7GB, 8GB	Linux, Windows
2048(2 vCPU)	4~16GB(1GB 증분)	Linux, Windows
4096(4 vCPU)	8~30GB(1GB 증분)	Linux, Windows
8192 (8 vCPU)	16~60GB(4GB 증분)	Linux
16384 (16vCPU)	32~120GB(8GB 증분)	Linux

- 리눅스 컨테이너 리소스 제한 : `ulimits`

[사전준비사항]

1) IAM : 아래 권한을 가진 사용자 IAM 사용자키(csv 파일) 생성후에 CLI 설정 (예시 사용자명 : EcsUser)

[AmazonEC2ContainerRegistryFullAccess](#), [AmazonElasticContainerRegistryPublicFullAccess](#),
[AmazonECS_FullAccess](#), [AmazonS3FullAccess](#)(파일다운로드),
[IAMFullAccess](#)(역할생성) , [CloudWatchFullAccess](#) (Log 그룹생성 삭제)

(익숙치 않을 경우 [AdministratorAccess](#) 권한 1 가지를 추가)

2) 작업 EC2 생성 (Name : ECS_Worker) Ubuntu 24.04 / X86-64 / t3.micro / 22 & 80 & 443 Port open 보안그룹

(ssh 접속 / console 혹은 ssh client - ex : putty)

```
sudo apt update -y
sudo apt install zip unzip -y
git clone https://github.com/GaussJung/aws-ecs.git
```

`cd ~/aws-ecs : docker_example 및 ecs_ref 디렉토리 유무 확인`

3) EC2 기동후 CLI 설정 (Linux-x86-64bit)

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip -qq awscliv2.zip
sudo ./aws/install
```

(이후 aws configure 진행 – IAM 사용자키 적용)

```
rm -rf aws*
```

4) 도커설정

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo apt-get install -y docker-compose
rm get-docker.sh
```

※ rootless 모드 도커활용

```
sudo apt-get install -y uidmap
dockerd-rootless-setuptool.sh install
```

(오류발생 예방을 위해 아래 2 개 진행)

```
sudo setcap cap_net_bind_service=ep $(which rootlesskit)
systemctl --user restart docker
```

5) 샘플 Docker 예제 Registry 등록

`cd ~/aws-ecs/docker_example/cityhome` : 명령이 실행되지 않는다면 git clone 먼저 진행 (위 EC2 생성 참조)
(이미지작성 및 구동)

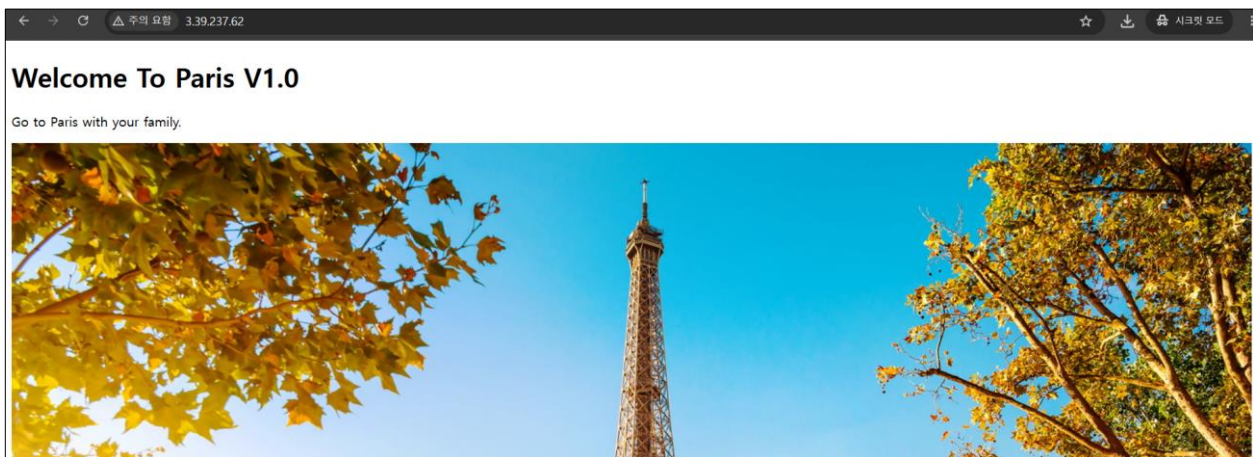
`docker compose up -d --build` : 파라미터 --build (도커이미지생성)

※ 위의 명령은 이미지작성과 컨테이너구동 동시에 진행

```
cd ~/aws-ecs/docker_example/cityhome/webapp_docker
```

```
docker build -t cityhome . && docker run -it -d -p 80:80 --name city80 cityhome
```

웹브라우저에서 퍼블릭 IP 혹은 DNS 접속하여 화면확인



[환경변수]

```
export myAwsAccount=111122223333 (※ 비고. 111122223333 은 콘솔우측 상단에 표기된 계정번호로 변경)
```

[ECR 생성]

```
aws ecr create-repository --repository-name cityhome --region ap-northeast-2
```

[로그인]

```
aws ecr get-login-password --region ap-northeast-2 | docker login --username AWS --password-stdin  
${myAwsAccount}.dkr.ecr.ap-northeast-2.amazonaws.com
```

[태깅]

```
docker tag cityhome:latest ${myAwsAccount}.dkr.ecr.ap-northeast-2.amazonaws.com/cityhome:latest
```

[푸시]

```
docker push ${myAwsAccount}.dkr.ecr.ap-northeast-2.amazonaws.com/cityhome:latest  
ubuntu@ip-172-31-18-91:~/aws-ecs/docker_example/cityhome$ docker push ${myAwsAccount}.dkr.ecr.ap-northeast-2.amazonaws.com/cityhome:latest  
The push refers to repository [569251118950.dkr.ecr.ap-northeast-2.amazonaws.com/cityhome]  
382a36159731: Pushed  
20d41cd6829d: Pushed  
f0a923958b1f: Pushed  
38a7e44929f3: Pushed  
83742381311a: Pushed  
8e9b53b630de: Pushed  
6e9f32fdbd61: Pushed  
1118cafd3938: Pushed  
6cea7312bacc: Pushed  
57fb71246055: Pushed  
latest: digest: sha256:84b9c1a4f508cb650243a04cd1ef638b22e9f3ae2e9c4a364b92af945d0053eb size: 856
```

6) IAM 설정

: 역할생성 (IAM 전체권한 혹은 관리자 권한 임시설정)

cd ~/aws-ecs/ecs_ref

ECS Task 실행 역할 생성 : ECS/Fargate 가 ECR 이미지 Pull, CloudWatch Logs 전송 등에 사용

```
aws iam create-role \
  --role-name myrole_ecsTaskExecution \
  --assume-role-policy-document file://ecs-tasks-trust-policy.json
```

ECS Task 역할 생성 : 컨테이너 내부 애플리케이션이 S3, DynamoDB, SQS 등 AWS API 호출 시 사용

```
aws iam create-role \
  --role-name myrole_ecsTaskRole \
  --assume-role-policy-document file://ecs-tasks-trust-policy.json
```

===== ecs-tasks-trust-policy.json 내용 =====

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowEcsTasksAssumeRole",
      "Effect": "Allow",
      "Principal": {
        "Service": "ecs-tasks.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

▶ myrole_ecsTaskExecution 정책붙임 (관리형정책)

```
aws iam attach-role-policy \
  --role-name myrole_ecsTaskExecution \
  --policy-arn arn:aws:iam::aws:policy/service-role/AmazonECSTaskExecutionRolePolicy
```

▶ myrole_ecsTaskRole 정책붙임 (인라인정책)

```
aws iam put-role-policy \
  --role-name myrole_ecsTaskRole \
  --policy-name mypolicy_ecsTaskAppAccess \
  --policy-document file://mypolicy_ecs_task.json
```

[AWS Fargate 콘솔 : Linux CLI - 사례 WebPage Example (Fargate Spot)]

CLI 참조 : <https://docs.aws.amazon.com/cli/latest/reference/ecs/index.html>

▶ STEP2-1. 클러스터 생성 :

```
aws ecs create-cluster --cluster-name myEcsSpotCluster --capacity-providers FARGATE
FARGATE_SPOT --region ap-northeast-2
```

```
{
  "cluster": {
    "clusterArn": "arn:aws:ecs:ap-northeast-2:5099511110950:cluster/myEcsSpotCluster",
    "clusterName": "myEcsSpotCluster",
    "status": "ACTIVE",
    "registeredContainerInstancesCount": 0,
    "runningTasksCount": 0,
    "pendingTasksCount": 0,
    "activeServicesCount": 0,
    "statistics": [],
    "tags": [],
    "settings": [
      {
        "name": "containerInsights",
        "value": "disabled"
      }
    ],
    "capacityProviders": [
      "FARGATE",
      "FARGATE_SPOT"
    ],
    "defaultCapacityProviderStrategy": []
  }
}
```

▶ STEP2-2. 테스크 생성

[Log 그룹생성 : 설정파일 myEcsWebServer-spot.json 의 logConfiguration 에서 참조됨]

```
aws logs create-log-group --log-group-name /ecs/myEcsSpotWebServer
```

[테스크정의]

(비고. myEcsWebServer-spot.json 파일의 11112222333 을 계정번호로 변경할것 - executionRoleArn 과 ECR Image repository 부분)

```
aws ecs register-task-definition --cli-input-json file://myEcsWebServer-spot.json
```

위의 작업은 반복시마다 태스크정의가 개정되어짐 (myEcsSpotWebFamily:1 --> myEcsSpotWebFamily:2)

※ 테스크삭제 (잘못만들어졌을 경우)

```
aws ecs deregister-task-definition --task-definition myEcsSpotWebFamily:1
aws ecs delete-task-definitions --task-definition myEcsSpotWebFamily:1
```

```
----- myEcsWebServer-spot.json -----
{
  {
    "family": "myEcsWebFamily",
    "networkMode": "awsvpc",
    "containerDefinitions": [
      {
        "name": "myEcsSpotWebServer",
        "image": "111122223333.dkr.ecr.ap-northeast-2.amazonaws.com/cityhome:latest",
        "cpu": 0,
        "portMappings": [
          {
            "containerPort": 80,
            "hostPort": 80,
            "protocol": "tcp"
          }
        ],
        "logConfiguration": {
          "logDriver": "awslogs",
          "options": {

```

```

        "awslogs-group": "/ecs/myEcsSpotWebServer",
        "awslogs-region": "ap-northeast-2",
        "awslogs-create-group": "true",
        "awslogs-stream-prefix": "ecs"
    },
    "healthCheck": {
        "retries": 3,
        "command": [
            "CMD-SHELL",
            "curl -f http://localhost:80/ || exit 1"
        ],
        "timeout": 5,
        "interval": 60,
        "startPeriod": 30
    },
    "essential": true
},
{
    "executionRoleArn": "arn:aws:iam::111122223333:role/myrole_ecsTaskExecutionRole",
    "requiresCompatibilities": [
        "FARGATE"
    ],
    "cpu": "256",
    "memory": "512",
    "runtimePlatform": {
        "cpuArchitecture": "X86_64",
        "operatingSystemFamily": "LINUX"
    },
    "tags": [
        {
            "key": "Name",
            "value": "myEcsSpotWebServer"
        }
    ]
}
]
}
----- EOF myEcsWebServer-spot.json -----

```


(실행결과 출력 : 생성된 Task 정의 --> 콘솔에서도 확인가능)

```
{
  "taskDefinition": {
    "taskDefinitionArn": "arn:aws:ecs:ap-northeast-2:111122223333:task-definition/myEcsSpotWebFamily:1",
    "containerDefinitions": [
      {
        "name": "myEcsSpotWebServer",
        "image": "111122223333.dkr.ecr.ap-northeast-2.amazonaws.com/cityhome:latest",
        "cpu": 0,
        "portMappings": [
          {
            "containerPort": 80,
            "hostPort": 80,
            "protocol": "tcp"
          }
        ],
        "essential": true,
        "environment": [],
        "mountPoints": [],
        "volumesFrom": [],
        "logConfiguration": {
          "logDriver": "awslogs",
          "options": {
            "awslogs-group": "/ecs/myEcsSpotWebServer",
            "awslogs-create-group": "true",
            "awslogs-region": "ap-northeast-2",
            "awslogs-stream-prefix": "ecs"
          }
        },
        "healthCheck": {
          "command": [
            "CMD-SHELL",
            "curl -f http://localhost:80/ || exit 1"
          ],
          "interval": 60,
          "timeout": 5,
          "retries": 3,
          "startPeriod": 30
        },
        "systemControls": []
      }
    ],
    "family": "myEcsSpotWebFamily",
    "executionRoleArn": "arn:aws:iam::111122223333:role/ myrole_ecsTaskExecutionRole",
    "networkMode": "awsvpc",
    "revision": 5,
    "volumes": [],
    "status": "ACTIVE",
    "requiresAttributes": [
      {
        "name": "com.amazonaws.ecs.capability.logging-driver.awslogs"
      },
      {
        "name": "ecs.capability.execution-role-awslogs"
      },
      {
        "name": "com.amazonaws.ecs.capability.ecr-auth"
      },
      {
        "name": "com.amazonaws.ecs.capability.docker-remote-api.1.19"
      },
      {
        "name": "ecs.capability.container-health-check"
      },
      {
        "name": "ecs.capability.execution-role-ecr-pull"
      },
      {
        "name": "com.amazonaws.ecs.capability.docker-remote-api.1.18"
      },
      {
        "name": "ecs.capability.task-eni"
      },
      {
        "name": "com.amazonaws.ecs.capability.docker-remote-api.1.29"
      }
    ],
    "placementConstraints": [],
    "compatibilities": [
      "EC2",
      "FARGATE"
    ],
    "runtimePlatform": {
      "cpuArchitecture": "X86_64",

```

```
        "operatingSystemFamily": "LINUX"
      },
      "requiresCompatibilities": [
        "FARGATE"
      ],
      "cpu": "256",
      "memory": "512",
      "registeredAt": "20XX-XX-04T20:13:03.018000+09:00",
      "registeredBy": "arn:aws:iam::111122223333:user/containeruser"
    },
    "tags": [
      {
        "key": "Name",
        "value": "myEcsSpotWebServer"
      }
    ]
  }
}
```

▶ STEP2-3. 서비스 생성

https://docs.aws.amazon.com/AmazonECS/latest/APIReference/API_CreateService.html

확인사항 : subnet-ID, 보안그룹-ID

```
aws ecs create-service \
  --cluster myEcsSpotCluster \
  --service-name myEcsSpotService \
  --network-configuration "awsvpcConfiguration={subnets=[subnet-bbaa91111,subnet-ddcc2222],securityGroups=[sg-aaaa1111bbbb],assignPublicIp=ENABLED}" \
  --cli-input-json file://myEcsSpotService.json
```

```
----- myEcsSpotService.json -----
{
  "taskDefinition": "myEcsSpotWebFamily:1",
  "desiredCount": 2,
  "capacityProviderStrategy": [
    {
      "base": 0,
      "capacityProvider": "FARGATE_SPOT",
      "weight": 1
    }
  ],
  "tags": [
    {
      "key": "Name",
      "value": "myEcsSpotService"
    },
    {
      "key": "Sector",
      "value": "MyTraining"
    }
  ]
}
```

ECS Service에서는 Capacity Provider Strategy를 사용하여 일반 Fargate와 FARGATE_SPOT을 혼합 운영할 수 있음
base 값은 최소 유지할 Task 수량을 의미하며, weight 값은 추가 Task를 어떤 비율로 분산 배치할지를 의미한다.

예를 들어 FARGATE의 weight=1, FARGATE_SPOT의 weight=3으로 설정하면 추가 Task는 약 1:3 비율로 배치됨.
이를 통해 안정성을 유지하면서도 운영 비용을 절감할 수 있음.

```
----- myEcsMixedService.json -----
{
  "taskDefinition": "myEcsMixedFamily",
  "desiredCount": 2,
  "capacityProviderStrategy": [
    {
      "capacityProvider": "FARGATE",
      "base": 1,
      "weight": 1
    },
    {
      "capacityProvider": "FARGATE_SPOT",
      "weight": 3
    }
  ],
  "tags": [
    {
      "key": "Name",
      "value": "myEcsMixedService"
    },
    {
      "key": "Sector",
      "value": "Production"
    }
  ]
}
```

비고) 서비스 삭제 (테스트수량조절 후에 삭제)

```
aws ecs update-service --cluster myEcsSpotCluster --service myEcsSpotService --desired-count 0
```

```
aws ecs delete-service --cluster myEcsSpotCluster --service myEcsSpotService
```

도커이미지를 새롭게 작성하여 **Push** 하였다면 **Task** 를 다시 등록하고 서비스 업데이트 진행

```
aws ecs register-task-definition  
--cli-input-json file://myEcsWebServer-spot.json
```

```
aws ecs update-service \  
--cluster myEcsSpotCluster \  
--service myEcsSpotService \  
--task-definition myEcsWebFamily
```

[콘솔에서 생성 확인]

(클러스터)

myEcsSpotCluster

Last updated
May 04, 2025 at 20:30 (UTC+9:00)

Update cluster

Delete cluster

Cluster overview

ARN
arn:aws:ecs:ap-northeast-2:5692511:18950:cluster/myEcsSpotCluster

Status
Active

CloudWatch monitoring
Default

Registered container instances
-

Services
Draining
-

Active
1

Tasks
Pending
-

Running
2

Services

Tasks

Infrastructure

Metrics

Scheduled tasks

Configuration

Tags

Services (1) Info

Manage tags

Update

Delete service

Create

Filter launch type
Any launch type

Filter service type
Any service type

Filter services by value

Service name

ARN

Status

Service type

Created at

myEcsSpotService

arn:aws:ecs

Active

REPLICA

6 minutes ago

(테스크정의)

Task definitions (1) Info

Last updated
May 04, 2025 at 20:31 (UTC+9:00)

Deploy

Create new revision

Create new task definition

Filter task definitions

Filter by status
Active

Task definition

Status of last revision

myEcsSpotWebFamily

ACTIVE

(서비스)

myEcsSpotService Info

Last updated
May 04, 2025 at 20:32 (UTC+9:00)

Update service

Delete service

Service overview Info

Status
Active

Tasks (2 Desired)
0 Pending | 2 Running

Task definition: revision
myEcsSpotWebFamily:5

Deployment status
Success

Health and metrics

Tasks

Logs

Deployments

Events

Configuration and networking

Service auto scaling

Tags

Tasks (1/2)

Filter desired status
Any desired status

Filter launch type
Any launch type

Filter tasks by property or value

Task

Last status

Desired st...

T...

Health sta...

Created at

Started by

Started at

17a19ff31a984438af260...

Running

Running

my...

Healthy

7 minutes ago

ecs-svc/23423958516...

6 minutes ago

adf5948d915c498caeb09...

Running

Running

my...

Healthy

6 minutes ago

ecs-svc/23423958516...

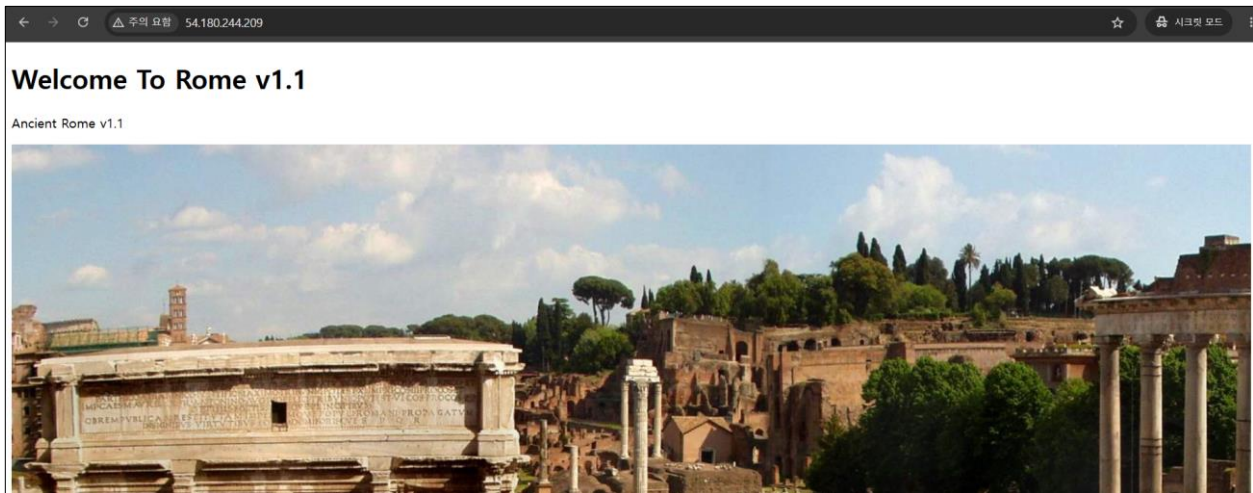
6 minutes ago

(Task 하단 컨테이너 정의)

Containers (1)							
Filter containers							
Container name	Container runtime ID	Image URI	Image Digest	Status	Health status	CPU	Memory hard
myEcsSpotWebServer	adf5948d915c498...	56925111...	sha256:72...	Running	Healthy	0	- / -

(Task 접속)

Task 에 할당된 Public IP 로 웹브라우저에 붙이고 접속 테스트



▶ 로드밸런서 + 서비스 AutoScaling 적용

사전작업 - 자원생성

1. 로드밸런서 생성

```
aws elbv2 create-load-balancer --name myECS-SVC-LB --subnets subnet-bde91111 subnet-bdbb2222 --security-groups sg-045490d9f04b67e3f --region ap-northeast-2
```

2. 목표그룹 생성

```
aws elbv2 create-target-group --name myECS-SVC-Group --target-type ip --protocol HTTP --port 80 --vpc-id vpc-75db461d --ip-address-type ipv4 --region ap-northeast-2
```

3. 리스너 생성

(80) 아래 로드밸런서 및 목표그룹 ARN 을 콘솔에서 확인하여 변경함. (붉은색상)

```
aws elbv2 create-listener --load-balancer-arn arn:aws:elasticloadbalancing:ap-northeast-2:111122223333:loadbalancer/app/myECS-SVC-LB/e72d1c914930efc0 --protocol HTTP --port 80 --default-actions Type=forward,TargetGroupArn=arn:aws:elasticloadbalancing:ap-northeast-2:111122223333:targetgroup/myECS-SVC-Group/b8be659e53280785
```

(443) 아래 인증서, 로드밸런서 및 목표그룹 ARN 을 콘솔에서 확인하여 변경함. (붉은색상)

```
aws elbv2 create-listener --load-balancer-arn arn:aws:elasticloadbalancing:ap-northeast-2:111122223333:loadbalancer/app/myECS-SVC-LB/e72d1c914930efc0 --protocol HTTPS --port 443 --certificates CertificateArn=arn:aws:acm:ap-northeast-2:111122223333:certificate/63ec49f5-717e-448f-89d1-49cfd95c9e3f --default-actions Type=forward,TargetGroupArn=arn:aws:elasticloadbalancing:ap-northeast-2:111122223333:targetgroup/myECS-SVC-Group/b8be659e53280785
```

(※ Public IP 할당해제 --> 이 경우 NAT Gateway 혹은 VPC EndPoint 가 설정되어 있어야 Task 동작함)

```
aws ecs update-service --cluster myEcsSpotCluster --service myEcsSpotService --network-configuration "awsvpcConfiguration={subnets=['subnet-bde986d5','subnet-bdbb67f1'],securityGroups=[sg-045490d9f04b67e3f],assignPublicIp=DISABLED}"
```

로드밸런서 적용

(참조 : <https://docs.aws.amazon.com/cli/latest/reference/ecs/update-service.html>)

[Task 수량 조절]

(4 개로 조절)

```
aws ecs update-service --cluster myEcsSpotCluster --service myEcsSpotService --desired-count 4
```

Tasks (1/4)							
Filter tasks by property or value				Filter desired status		Filter launch type	
Filter tasks by property or value				Any desired status		Any launch type	
Task	Last status	Desired st...	Task d...	Health sta...	Created at	Started by	
 c9bcb99029d4deba62b8...	 Provisioning	 Running	myEcsSp...	 Unknown	8 seconds ago	ecs-svc/27752284087...	
 e7c7c7c188704033801dd...	 Running	 Running	myEcsSp...	 Healthy	18 minutes ago	ecs-svc/27752284087...	
 eec2bc99184148d5a28b1...	 Running	 Running	myEcsSp...	 Healthy	18 minutes ago	ecs-svc/27752284087...	
 f4f82be524644cb7a22eb...	 Provisioning	 Running	myEcsSp...	 Unknown	8 seconds ago	ecs-svc/27752284087...	

(0 개로 조절 : 즉, Task 작동 중지)

```
aws ecs update-service --cluster myEcsSpotCluster --service myEcsSpotService --desired-count 0
```

[서비스에 로드밸런서 적용 + Task 수량 3 개로 조정]

```
aws ecs update-service --cluster myEcsSpotCluster --service myEcsSpotService --desired-count 3
--load-balancers "targetGroupArn=arn:aws:elasticloadbalancing:ap-northeast-
2:111122223333:targetgroup/myECS-SVC-
Group/b8be659e53280785,containerName=myEcsSpotWebServer,containerPort=80"
```

[서비스에 로드밸런서 적용 (수량조절없이)]

```
aws ecs update-service --cluster myEcsSpotCluster --service myEcsSpotService --load-balancers
"targetGroupArn=arn:aws:elasticloadbalancing:ap-northeast-2:111122223333:targetgroup/myECS-SVC-
Group/b8be659e53280785,containerName=myEcsSpotWebServer,containerPort=80"
```

[로드밸런서 제거]

```
aws ecs update-service \
  --cluster myEcsSpotCluster \
  --service myEcsSpotService \
  --load-balancers '[]'
```

[로드밸런서 PublicDNS 혹은 인증서 설정 SSL URL 로 접속함]

--> Task 로 접속했을 때와 동일한 화면 출력

myECS-SVC-Group

Actions

Details

arn:aws:elasticloadbalancing:ap-northeast-2:569251118950:targetgroup/myECS-SVC-Group/b8be659e53280785

Target type

IP

Protocol : Port

HTTP: 80

Protocol version

HTTP1

VPC

[vpc-75db461d](#)

IP address type

IPv4

Load balancer

[myECS-SVC-LB](#)

2

Total targets

2

Healthy

0

Anomalous

0

Unhealthy

0

Unused

0

Initial

0

Draining

► Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets

Monitoring

Health checks

Attributes

Tags

Registered targets (2)

Info

Anomaly mitigation: Not applicable

Deregister

Register targets

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Filter targets

< 1 >

	IP address	Port	Zone	Health status	Health status details	Administrative override	Overri...
☐	172.31.1.221	80	ap-northea...	Healthy	-	No override	No overri...
☐	172.31.17.175	80	ap-northea...	Healthy	-	No override	No overri...

16 / 18

▶ 서비스 오토스케일링 정책 적용

참조) <https://repost.aws/knowledge-center/ecs-fargate-service-auto-scaling>

[오토스케일링]

- (선택) Task Definition 이 업데이트된 경우 서비스에 최신 Revision 반영

```
aws ecs update-service \  
  --cluster myEcsSpotCluster \  
  --service myEcsSpotService \  
  --task-definition myEcsWebFamily
```

- 사전수행 0 개로 변경 (필요시 --> 단, 권장사항 아님)

```
aws ecs update-service \  
  --cluster myEcsSpotCluster \  
  --service myEcsSpotService \  
  --desired-count 0
```

A1) 어플리케이션 오토스케일링 타겟 설정 (2 개 ~ 6 개)

- 중요사항 : Resource-ID : 클러스터명 + 서비스명

```
aws application-autoscaling register-scalable-target \  
  --service-namespace ecs --scalable-dimension ecs:service:DesiredCount \  
  --resource-id service/myEcsSpotCluster/myEcsSpotService \  
  --min-capacity 2 --max-capacity 6 --region ap-northeast-2
```

(결과 예시)

```
{  
  "ScalableTargetARN": "arn:aws:application-autoscaling:ap-northeast-2:111122223333:scalable-  
target/0ec5de7fd7704c4548278e1e60d617278376"  
}
```

A2) 추적정책등록-CPU (CPU 이용률 50% 초과시 자원증대, 확장시 70 초간 확인, 수축시 60 초 확인)

```
aws application-autoscaling put-scaling-policy \  
  --service-namespace ecs \  
  --scalable-dimension ecs:service:DesiredCount \  
  --resource-id service/myEcsSpotCluster/myEcsSpotService \  
  --policy-name my-ecs-scaling-policy \  
  --policy-type TargetTrackingScaling \  
  --target-tracking-scaling-policy-configuration  
'{"TargetValue":50.0,"PredefinedMetricSpecification":{"PredefinedMetricType":"ECSServiceAverageC  
PUUtilization"},"ScaleOutCooldown":70,"ScaleInCooldown":60}' \  
  --region ap-northeast-2
```

(결과예시)

```
{  
  "PolicyARN": "arn:aws:autoscaling:ap-northeast-2:111122223333:scalingPolicy:de7fd770-4c45-4827-8e1e-  
60d617278376:resource/ecs/service/myEcsSpotCluster/myEcsSpotService:policyName/my-ecs-scaling-policy",  
  "Alarms": [  
    {  
      "AlarmName": "TargetTracking-service/myEcsSpotCluster/myEcsSpotService-AlarmHigh-7ed79954-173f-435e-  
9597-98965851f7ff",  
      "AlarmARN": "arn:aws:cloudwatch:ap-northeast-2:111122223333:alarm:TargetTracking-  
service/myEcsSpotCluster/myEcsSpotService-AlarmHigh-7ed79954-173f-435e-9597-98965851f7ff"  
    },  
    {  
      "AlarmName": "TargetTracking-service/myEcsSpotCluster/myEcsSpotService-AlarmLow-a17e9386-2a58-4ca3-  
b9ed-459821a36aca",  
      "AlarmARN": "arn:aws:cloudwatch:ap-northeast-2:111122223333:alarm:TargetTracking-  
service/myEcsSpotCluster/myEcsSpotService-AlarmLow-a17e9386-2a58-4ca3-b9ed-459821a36aca"  
    }  
  ]  
}
```

[콘솔 확인]

The screenshot shows the AWS Management Console for the service 'myEcsSpotService'. The status is 'Active'. There are 2 desired tasks, with 0 pending and 2 running. The deployment status is 'Success'. The 'Service auto scaling' tab is selected, showing a task count of 2-6. A scaling policy named 'fun-ecs-scaling-policy' is listed, using 'Target Tracking' with a target of 'ECSServiceAverageCPUUtilization (45%)'.

[자원삭제]

: ECS Task 및 서비스삭제 (콘솔 삭제 혹은 아래 CLI)

```
aws ecs update-service --cluster myEcsSpotCluster --service myEcsSpotService --desired-count 0
aws ecs delete-service --cluster myEcsSpotCluster --service myEcsSpotService
```

: LoadBalancer 삭제 (콘솔 삭제 혹은 아래 CLI)

```
aws elbv2 delete-load-balancer --load-balancer-arn arn:aws:elasticloadbalancing:ap-northeast-2:111122223333:loadbalancer/app/myECS-SVC-LB/7253f29426aec20b
```

: Target Group 삭제 (콘솔 삭제 혹은 아래 CLI)

```
aws elbv2 delete-target-group --target-group-arn arn:aws:elasticloadbalancing:ap-northeast-2:111122223333:targetgroup/myECS-SVC-Group/e500d2afeea00c58
```

: 오토스케일링 정책제거

```
aws application-autoscaling deregister-scalable-target \
  --service-namespace ecs \
  --scalable-dimension ecs:service:DesiredCount \
  --resource-id service/myEcsSpotCluster/myEcsSpotService \
  --region ap-northeast-2
```

: ECS Task 등록해제(수량만큼)

```
aws ecs deregister-task-definition --task-definition myEcsWebFamily:1
aws ecs deregister-task-definition --task-definition myEcsWebFamily:2
aws ecs deregister-task-definition --task-definition myEcsWebFamily:3
```

: ECS Task 삭제 (수량만큼)

```
aws ecs delete-task-definitions \
  --task-definitions \
  myEcsWebFamily:1 \
  myEcsWebFamily:2 \
  myEcsWebFamily:3 \
```

: ECS Cluster 삭제

```
aws ecs delete-cluster --cluster myEcsSpotCluster
```

: Log 그룹삭제

```
aws logs delete-log-group --log-group-name /ecs/myEcsSpotWebServer
```

- 끝 -